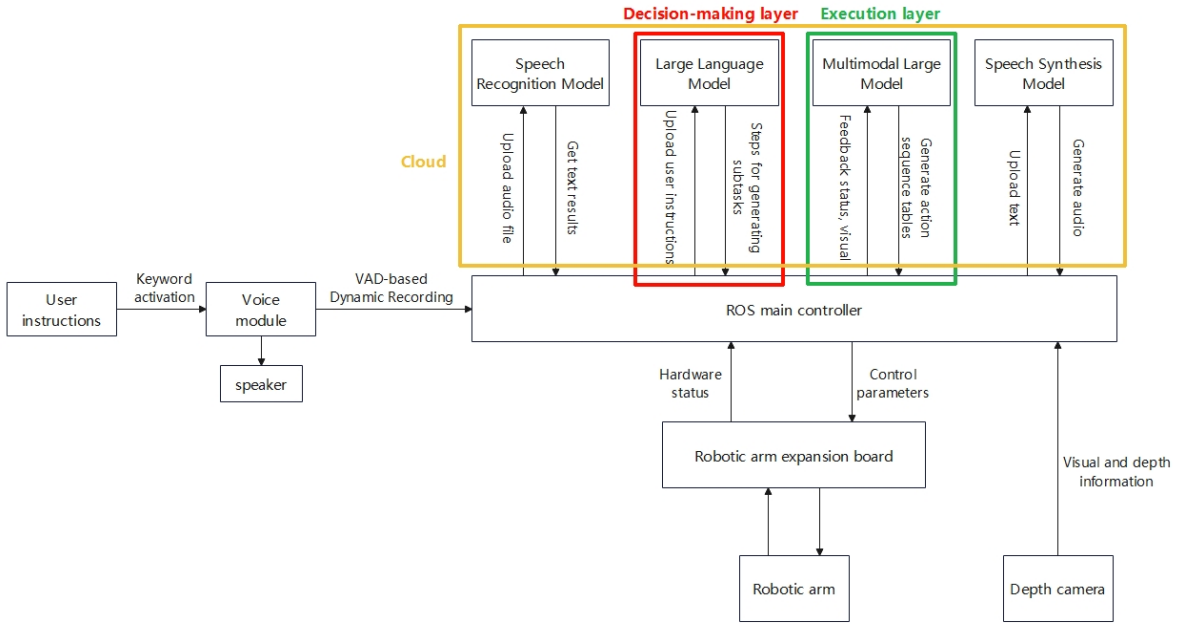
Embodied Intelligent Robot System Architecture

Course Content

1. Explain the large model inference architecture for AI embodied intelligent robots
2. Explain basic concepts in the system to lay the groundwork for hands-on exercises in subsequent courses

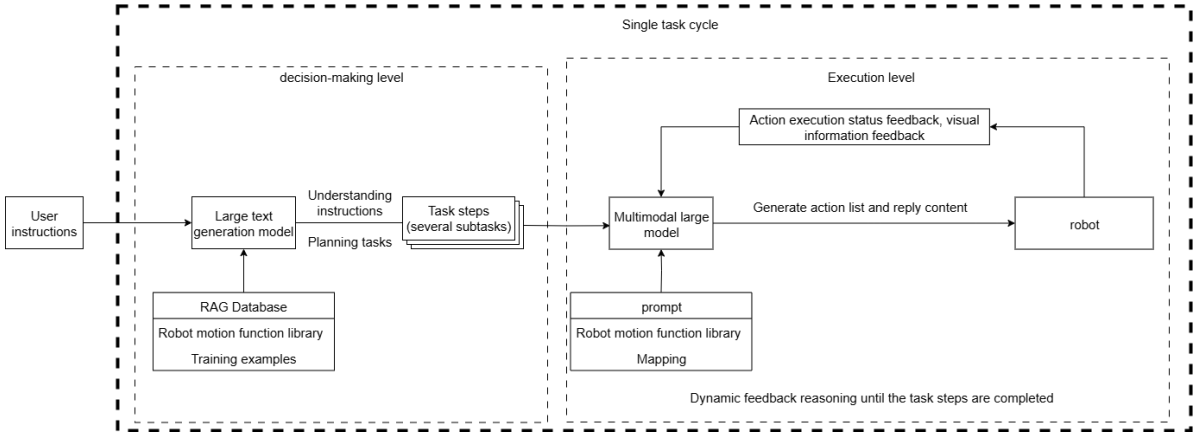
System Composition

Dofbot-SE is not equipped with a depth camera; the USB camera only provides color images



AI Large Model Inference Architecture Diagram

The robot adopts a **dual-model inference** and **dynamic feedback inference** design in its programming, which improves the robot's ability to handle long-process complex tasks and provides stronger system robustness compared to single-model architectures. There are several key concepts here: decision-layer large model, execution-layer large model, task cycle, and conversation history, which will be explained one by one below.



Large Model Inference Architecture Explanation

4.1 Dual-Model Architecture Principles

- The robot's large model control system is designed based on text-generation large models and multimodal large models. The text-generation large model serves as the **decision layer**, functioning as an upper-level task planner to decompose complex, abstract human instructions into task steps
- The multimodal large model serves as the execution layer, receiving **task steps generated by the decision layer** and user's **temporary instructions** (generally simple instructions requiring the robot to end tasks or rest), supervising the robot's task execution progress, making real-time adjustments to execution actions, generating a list of action functions that the robot can parse, and generating responses to the user
- Among these, the action function list contains pre-written basic action functions that can directly output parameters for controlling robot movement.

4.2 Advantages of Dual-Model Inference Architecture

4.2.1 Decoupling of Decision Logic and Motion Control

- The multimodal large model is inferior to the text-generation large model in natural language semantic understanding and logical reasoning capabilities, and is prone to modality interference when executing complex tasks. Therefore, the system design decouples decision logic from motion control.
- The text-generation large model focuses on semantic understanding and task planning (for example, parsing "Can you help me get the blue block in room A?" and decomposing it into task steps: "Record current position → Navigate to room A → Get current robot perspective image → Locate blue block coordinates → Grasp blue block → Return to starting position → Place down blue block"), avoiding the complexity that traditional single models face when simultaneously processing action details during semantic parsing.
- The multimodal large model is responsible for action generation and environmental interaction (for example, obtaining visual images to locate object coordinates, outputting action functions and parameters), reducing the confusion of action logic and significant deviations caused by task complexity in single models.

4.2.2 Reducing Model Training Sample Complexity

When using a single model for inference, the multimodal large model needs to simultaneously perform "natural language understanding + environmental perception + process decomposition + action function output," which can easily lead to **modality interference** (language ambiguity or overly long instructions causing comprehension errors). The dual model processes tasks in stages, allowing the decision layer to focus on language space mapping while the execution layer focuses on visual-motor space mapping.

4.2.3 Flexible Expansion

The large models for the decision layer and execution layer can be flexibly combined with various large models available on the Ali Bailian large model platform (sometimes also called the Tongyi Qianwen platform). Through custom training samples and RAG knowledge bases, models can be adapted to tasks in different scenarios, greatly enhancing the generalization capability of AI embodied intelligent robots across various scenarios.

4.3 Decision-Layer Large Model

4.3.1 Role of the Decision-Layer Model

- The robot's decision-layer large model uses the Tongyi Qianwen-Max series model by default.
- The decision-layer large model is primarily responsible for task planning. It can understand complex human instructions and decompose them into specific task steps.
- Each task step corresponds to the smallest action the robot can perform. The execution-layer large model then implements specific robot movements by calling API functions from the robot's action function library.

4.3.2 Robot Action Function Library (Simplified)

The robot action library specifies all the smallest actions the robot can actually perform. When planning task steps, the decision-layer model selects appropriate actions from it and arranges them in combinations.

Basic Action Class

- Turn on red light
- Turn on green light
- Turn on blue light
- Turn on alarm
- Turn off alarm
- Turn off lights

Arm Class

- Adjust servo #X to Y degrees

  Adjust the specified servo to rotate to the specified angle
- Arm movement
  Description: The arm end moves in the specified direction by the specified distance
- Arm moves upward
- Arm nods
- Arm shakes head
- Arm dances
  Similar semantics: shake head, sway head as indication.
- Arm applauds
  Similar semantics: applaud, clap as indication.
- Arm grasps, picks up object
  Similar semantics: pick up, grab
- Arm places down object
  Description: The arm releases the grasped object
- Arm gripper opens
  Description: Adjust the angle of servo #6 to 0 degrees
- Arm gripper closes
  Description: Adjust the angle of servo #6 to 180 degrees
- Sort, grasp machine code #X
  Description: Sort and grasp the specified machine code number.

- Track object, description: Track the specified object
- Sort trash of type X
  Description: Call the trash sorting function to sort a specific type of trash. X can be 'rec', 'tox', 'wet', or 'dry'.
- Move A to the side of B
  Description: Change the spatial position of grasping object A to the left side of object B. Parameters: (x1, y1) are the pixel coordinates of the upper-left corner of object A's outer border, (x2, y2) are the pixel coordinates of the lower-right corner of object A's outer border, (x3, y3) are the pixel coordinates of the upper-left corner of object B's outer border, (x4, y4) are the pixel coordinates of the lower-right corner of object B's outer border, src represents the name of object A, if A is a color block, src represents the color of the color block, tar represents the name of object B, similarly, if B is a color block, tar represents the color of the color block; side represents the five directions: front, back, left, right, top. The corresponding values are: 1 for front, 2 for back, 3 for left, 4 for right, 5 for top.
  Similar semantics: grasp A to the side of B, place A to the side of B, put A on the side of B
- Color block stacking
  Description: Grasp the color blocks in the image according to the given order.
- Place the color block back to its original position
  Description: Place the color block back to its original position
  Similar semantics: return the color block to its original position, place the color cube back to its original position.
- Track object
- Track machine code
- Cancel tracking machine code
- Cancel tracking object
- Remember the current position of the object
- Arm pointing
- Trash sorting
- Trash cleanup
- Sort color blocks according to the given sorting order

Image Capture Class

- Get current perspective image
  Description: Get the current perspective image for observing the environment or locating items.
- Record X seconds of video
  Description: Record video of the specified duration. X represents the duration in seconds.
- Play a guessing game/ball game
  Description: Record 20 seconds of video.
- Video understanding
  Description: Parse the content of the recorded 20-second video

Other Class

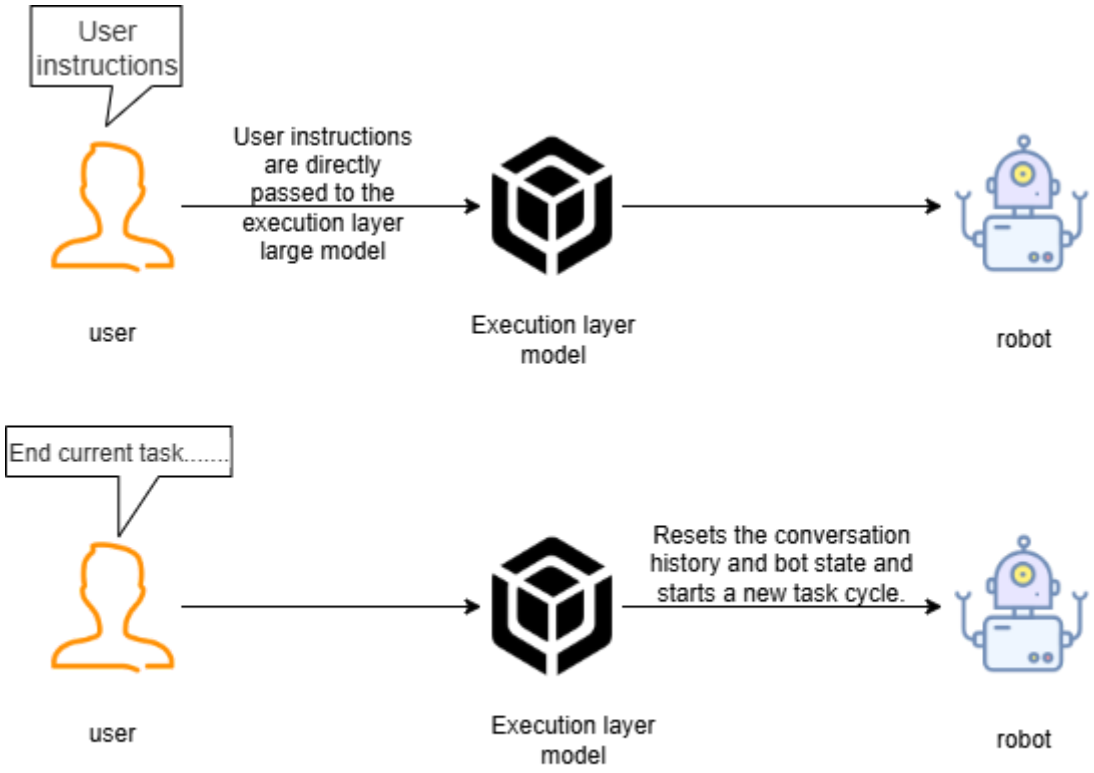
- Query weather
- Calculate math problems
- Guess object in English
- Tell a joke

4.4 Execution-Layer Large Model

- The robot's execution-layer large model uses the Tongyi Qianwen-VL series model by default.
- The execution-layer large model is primarily responsible for generating the action list that controls the robot's behavior and for responding to the user. During the robot's execution of the action list, it continuously receives feedback from the robot's action execution results (success/failure), visual images, and based on the success or failure of the action execution, infers the next action to be executed
- The execution-layer large model acts as a supervisor, continuously monitoring the progress of the robot's task execution. Based on the robot's action execution results and environmental information, it thinks and judges the next action to execute until the task is successfully completed or ended early due to special circumstances.

4.4.1 Task Cycle

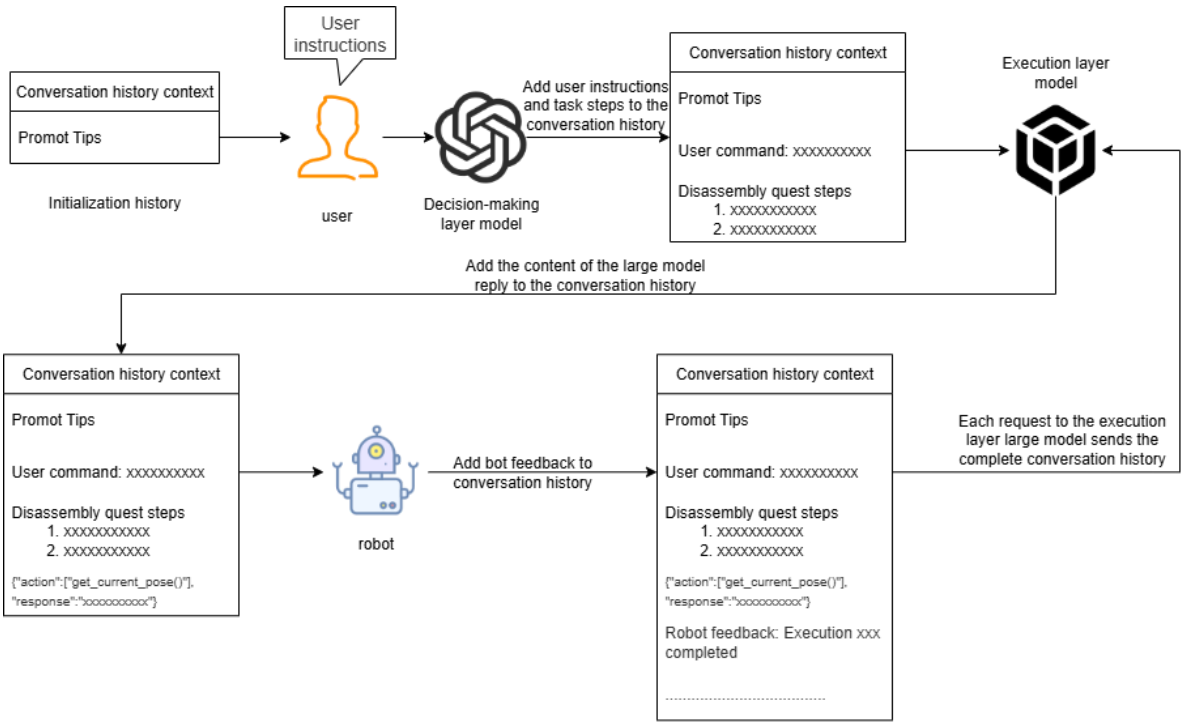
- In a new task cycle, the user's instruction first passes through the decision-layer large model, then through the execution-layer large model. If the execution-layer large model determines that the robot has completed all task steps, it enters the **waiting state**. At this time, subsequent user instructions are directly handed to the execution-layer large model.
- For example: When the user requests to end the current task and let the robot rest, the robot will reset the conversation history and arm state, starting a new task cycle. At this time, the user's instruction will start from the decision-layer large model. The following is a diagram of the waiting state:



The waiting state of the robot after completing the task

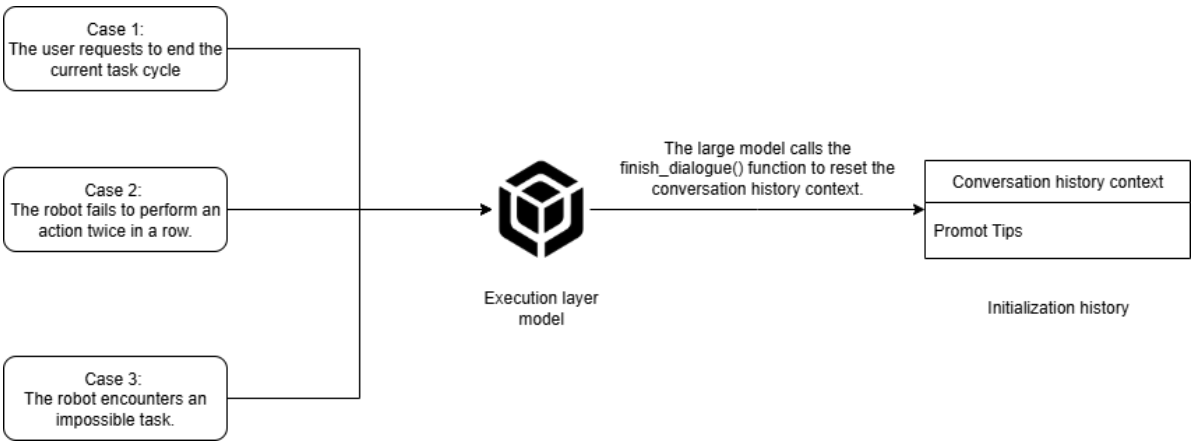
4.4.2 Historical Context

The robot maintains a conversation history context locally in its program. The conversation history context contains the user's instructions, task steps generated by the decision-layer model, action lists generated by the execution-layer model, and status information fed back by the robot. Each time the robot requests the execution-layer large model, it sends the complete conversation history. Therefore, the execution-layer large model can know the robot's task execution progress. The execution-layer large model combines the conversation history context to infer the next action to execute. In summary, **the execution-layer large model derives future next actions based on the sum of past states**.



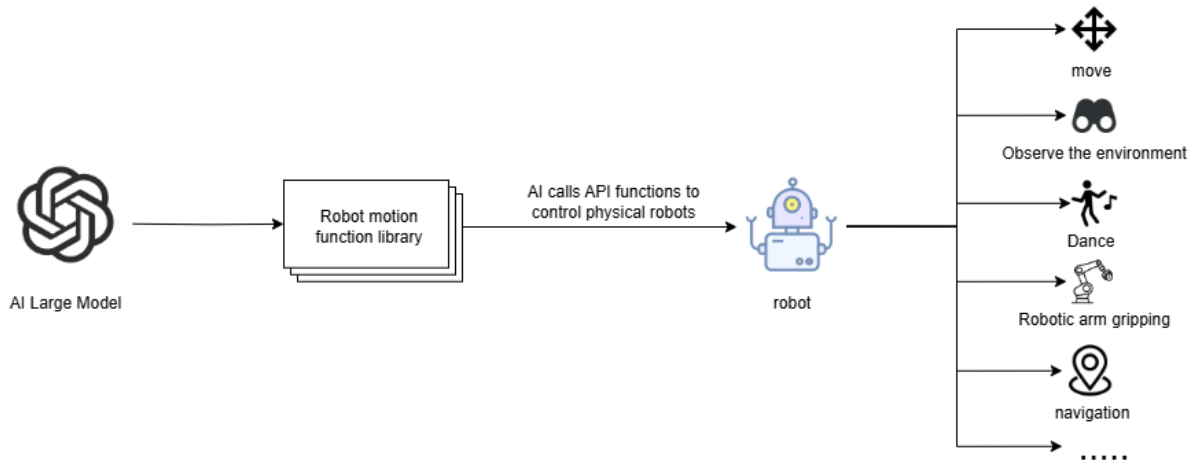
Under normal circumstances, the execution-layer large model determines the progress of task completion based on the conversation history. When it determines that the robot has completed all task steps, it enters the **waiting state**, waiting for the user's temporary instructions. Temporary instructions will not go through the decision-layer large model again but are directly passed to the execution-layer large model. There are three situations that will cause the robot to clear the conversation context history, end the current task cycle, and start a new task cycle.

- Situation 1: The user actively requests to end the task cycle. In the **waiting state**, when the user says something like "You can rest now" or "You may withdraw," indicating the robot is no longer needed, the execution-layer large model will call the finish_dialogue() function to end the current task cycle and reset the robot's conversation history, starting a new task cycle. The same applies to the following situations.
- Situation 2: The robot fails the same action twice consecutively. If an action fails, the execution-layer large model will control the robot to retry at most once. If it fails again, the execution-layer large model will control the robot to end the task cycle early and start a new task cycle.
- Situation 3: The robot encounters an impossible task. For example, when the user asks the robot to go to a location not mentioned in the "map mapping," the robot will request to end the task cycle early and inform the user that it cannot complete this task.



4.4.4 Robot Action Function Library

The API functions in the robot action function library serve as a bridge for the large model to control the robot's interaction with the real world. These API functions specify the smallest actions that the physical robot can perform in the physical world. The principle of API functions is to control the robot's underlying hardware through the ROS2 system to complete various functions.



All action functions and their corresponding functions are shown in the following table:

Function Name	Parameters	Function	Call Command
arm_dance()	-	Trigger arm dance action	Arm dances
arm_up()	-	Arm moves upward	Arm up
arm_down()	-	Arm moves downward	Arm down
arm_nod()	-	Arm nods	Nod
arm_shake()	-	Arm shakes head	Shake head
arm_applaud()	-	Arm applauds	Applaud
grip_pose()	-	Arm grasp object pose	Arm adjusts to grasp pose
track_pose()	-	Arm track object pose	Arm adjusts to track pose
cancel_KCF_follow	-	Cancel object tracking	Cancel object tracking
grasp_obj(x1, y1, x2, y2)	(x1, y1, x2, y2) Coordinates of the upper-left and lower-right points of the target object's outer border	Arm grasps a certain object	Grasp xxx
putdown()	-	Arm places down the grasped object	Put down xxx
apriltag_sort(x)	x: Machine code number	Sort out the specified machine code	Sort machine code #x
track(x1, y1, x2, y2)	(x1, y1, x2, y2) Coordinates of the upper-left and lower-right points of the target object's outer border	Track the specified object within the field of view	Track xx
garbage_sort(type_)	type_: Type of garbage to sort	Sort specified type of garbage	Sort recyclable garbage

Function Name	Parameters	Function	Call Command
change_pose(x1, y1, x2, y2,x3,y3,x4,y4,src,tar,side)	x1, y1, x2, y2, x3, y3, x4, y4: Represent the coordinates of the upper-left and lower-right corners of the top outer borders of the target transport color block and target placement color block respectively; src represents the color of the target transport color block, tar represents the color of the target placement color block, side represents the placement direction	Transport the color block to the direction of another color block	Place the blue block on top of the yellow block
record_video(time_)	time_ represents the duration of video recording	Record video	Record a video of x seconds
apriltag_follow(target_id)	target_id: Machine code ID	Track the machine code with the specified ID	Track machine code #x
compute_pose(x1,y1,x2,y2,name)	x1, y1, x2, y2: Coordinates of the upper-left and lower-right corners of the outer border of the color block to be recorded, name represents the color of the color block to be recorded	Record the current position of the specified color block	Record the current position of the x color block

Function Name	Parameters	Function	Call Command
compute_pose_order(x1,y1,x2,y2,name,order)	x1, y1, x2, y2: Coordinates of the upper-left and lower-right corners of the outer border of the top color block, name: color of the top color block, order represents the stacking order of color blocks from top to bottom	Record the position of each color block in the stacked color blocks	Record the current position of color blocks on the table
return_to_orin(name)	name: Color of the color block to be returned	Place the specified color block back to its original position	Place the x color block back to its original position
light_on(color)	color: LED light color	Turn on LED light	Turn on * color light
light_off()	-	Turn off LED light	Turn off lights
beep_on()	-	Turn on buzzer	Buzzer alarm
beep_off()	-	Turn off buzzer	Turn off buzzer
adjust_joint(joint_id,angle)	joint_id: Servo ID, angle: Servo angle	Set servo angle	Set servo #x to y degrees
gripper_open()	-	Open gripper	Open gripper
gripper_close()	-	Close gripper	Close gripper
color_back_to_orin()	-	Publish color block return order	Put the color blocks on the table back to their original positions
grasp_from_rm_list(color)	color: Color block color	Grasp color block from the grasp list	Grasp the x color block from the grasp list

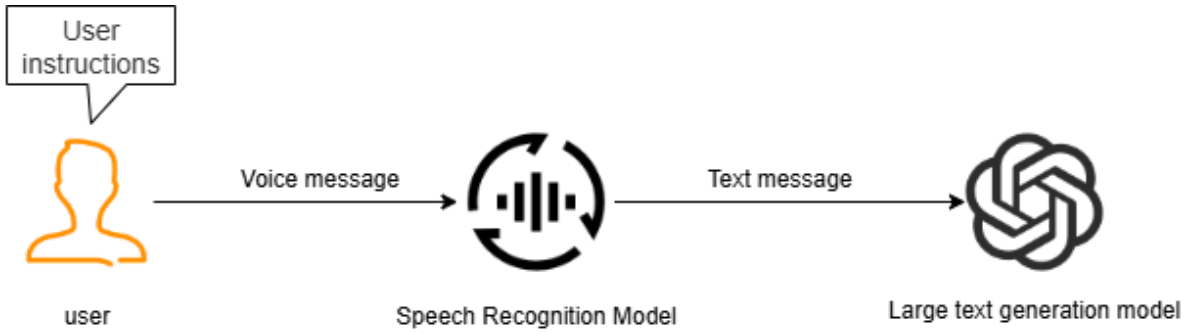
Function Name	Parameters	Function	Call Command
arm_move(Dir,Dist)	Dir: Direction for the arm end adjustment, values are up, down, left, right, front, back; Dist: Distance for the arm end adjustment in centimeters	Adjust the position of the arm end	Arm adjusts by x in the y direction by z centimeters
point_to(x1, y1, x2, y2)	x1, y1, x2, y2: Coordinates of the upper-left and lower-right corners of the object's outer border that is being pointed to	Arm points to object	Arm points to x
garbage_sort_all()	-	Trash tag code sorting	Clean up trash on the table
grasp_from_down_list(color)	color: Color block color	Grasp color block from the placement list	Grasp the x color block from the placement list
check_remove(color)	-	Check if the specified color block exists in the removal list	Check if the x color block exists in the removal list
seewhat()	-	Take a photo of the robot's current perspective and upload it to the execution-layer large model	Observe environment
video_understanding()	-	Understand video content	What is in the video that was just recorded
finish_dialogue()	-	Reset historical context and start a new task cycle	-

Function Name	Parameters	Function	Call Command
wait(x)	x: Time in seconds	Wait for a period of time without performing any operations	Wait x seconds
finishtask()	-	Automatically called after the robot completes a task, used to stop feeding status back to the execution-layer model	-

5. Application of Voice Model in the System

5.1 Voice Recognition

Since the decision-layer and execution-layer large models are respectively a text-generation model and a visual multimodal model, they cannot directly receive the user's voice information. Therefore, a voice recognition large model is needed to convert the user's voice instructions into corresponding text, which is then handed to the AI large model.



5.2 VAD Voice Activity Detection

VAD (Voice Activity Detection) is a technology that can automatically identify speech segments and non-speech segments (such as silence and noise) in voice signals. It locates the starting and ending positions of speech from the audio stream and filters out invalid background sounds.

In subsequent hands-on courses, when the user uses the wake word "Hi,yahboom" to wake up the robot, the system will enter VAD voice activity detection. The system will automatically detect the user's speaking duration and save valid sound fragments as WAV audio files. The audio files will then be given to the voice recognition model to convert them into text.

5.3 Speech Synthesis

The text response generated by the large model for the user is converted into audio through the speech synthesis model, which is then played by the hardware speaker.